

Part 2: Lecture 1

TECH2: Introduction to Programming, Data, and Information Technology

Richard Foltyn

NHH Norwegian School of Economics

September 30, 2026

See GitHub repository for notebooks and data:

<https://github.com/richardfoltyn/TECH2-H26>

Contents

- 1 Python ecosystem
- 2 Outline of Part 2
- 3 Goals for today
 - GitHub & tools
 - Set up local TECH2-H26 repository
 - Set up TECH2 Conda environment
 - Implement and benchmark `argmax()`
 - Debugging in VS Code
- 4 Additional resources

About me

- Undergraduate studies in software engineering (& economics), PhD in Economics
- Research fields: Quantitative Macroeconomics & Household Finance
- 20+ years of programming experience:
 - Previously (and mostly forgotten): C/C++, Visual Basic, Java, JavaScript, PHP, Perl, SQL, Matlab, R
 - These days: Python, Fortran, Unix shell scripts, Stata

Contact

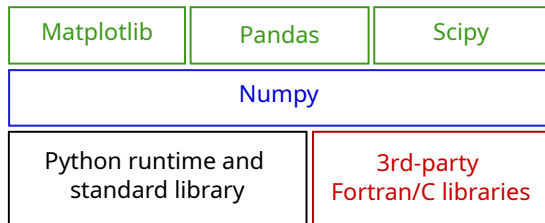
- Email: richard.foltyn@nhh.no
- Office: D231 (SAM, 2nd floor in the new building)

PYTHON ECOSYSTEM

Python software stack

How things fit together

- “Python” is the language & standard library supported by the [Python Software Foundation](#)
- For numerical applications, we need additional third-party packages such as [NumPy](#), [SciPy](#), etc.
- For working with data, we use additional packages such as [Pandas](#)



Python software stack for data science

Used in this course

- **Python**: Language, runtime, and standard libraries (“Python”)
- **NumPy**: Implements n -dimensional arrays, linear algebra routines, and random number generators
- **Matplotlib**: High-level plotting routines for data visualization
- **Pandas**: Containers for handling heterogeneous data & routines for data analysis
- **Jupyter notebooks**: Interactive documents that contain code, figures, and text

Not used in this course (but useful for future work)

- **SciPy**: Optimization routines, sparse matrices, integration, interpolation, linear algebra, statistics
- **statsmodels**: Estimation of (mostly linear) econometric & statistical models
- **scikit-learn**: Routines used for machine learning (Ridge regression, Lasso, elastic net, etc.)

OUTLINE OF PART 2

Outline of Part 2

Week 40: GitHub & NumPy

- GitHub basics (cloning & forking, pulling & pushing)
- Programming concepts: Vectorization vs. loops
- Application: Write and benchmark the `argmax()` function

Week 41: Introduction to pandas

- Creating/importing data
- Data cleaning
- Summary statistics
- Indexing
- Time series data

Outline of Part 2 (continued)

Week 42: Plotting

- Creating line plots, bar charts, etc.
- Plotting data with pandas

Week 43: Data wrangling I

- Aggregation and reduction
- Transformations
- Resampling of time series data

Week 44: Data wrangling II

- Concatenating data
- Merging & joining data

Outline of Part 2 (continued)

Week 45: AI week

- Wednesday, Nov 4, 8:15–12:00
 - Overview of the AI landscape (vendors, subscriptions, API access)
 - Overview of developer tools (integrated in VS Code, command line)
 - Overview of coding agents:
Claude Code, OpenAI Codex, Google Antigravity CLI, OpenCode, Pi
 - Hands-on exercises with coding agents
- Friday, Nov 6, 12:15–16:00
 - Presentation by KPMG on how they integrate AI into customer deployments
 - Some social activity afterward

TECH2 is the foundation for other courses in BEDS

Semester 2

1 TECH3: Applied Statistics

- Uses Python as the programming language for statistical analysis

Semester 3

2 TECH4: Mathematics for Decisions

- Python optimizers and solvers

Semester 5

3 TECH5: Programming and Application Design

- Object-oriented programming with Python

4 TECH6: Data Science and Econometrics

- Machine Learning (ML) with Python

+ any other task that requires you to work with data

GOALS FOR TODAY

Goals for today

- 1 GitHub & other tools
- 2 Get the TECH2-H26 repository up and running
 - Clone or fork the repository at <https://github.com/richardfoltyn/TECH2-H26>
- 3 Create the TECH2 environment (if you did not do so last week)
- 4 Implement a function `argmax()` and benchmark it against NumPy's implementation.
- 5 Explore debugging options in VS Code (if there is time)
- 6 Optional: Integrate Git into your programming workflow
 - Add commits as you finish individual tasks
 - Push local commits to GitHub once you're done

GOAL 1: GITHUB & TOOLS

Git & GitHub?

Why git? (and GitHub)

- Because everyone uses it: it has almost completely replaced all other version control systems over the last 20 years

Examples:

- Python: <https://github.com/python/cpython>
- NumPy: <https://github.com/numpy/numpy>
- SciPy: <https://github.com/scipy/scipy>
- Pandas: <https://github.com/pandas-dev/pandas>
- Matplotlib: <https://github.com/matplotlib/matplotlib>
- PyTorch (Meta's ML library): <https://github.com/pytorch/pytorch>
- TensorFlow (Google's ML library): <https://github.com/tensorflow/tensorflow>
- Keeps history of **your** code changes (and can restore previous versions)
- Keeps history of **others'** code changes
- Allows for decentralized coding in teams
- Allows synchronizing code across devices

Why GitHub?

- Almost everyone uses it (for now)
- Alternatives (less popular):
 - [GitLab](#)
 - [Bitbucket](#)
 - [Codeberg](#) (only for open-source projects)
- Offers many services beyond version control (issue tracking, wikis, etc.)
- Includes some (limited) AI credits in education plans that can be used with GitHub Copilot
- Register for free at <https://github.com/signup>

- Sign up once you have a GitHub account: <https://github.com/education/students>
- Benefits: unlimited private repositories, GitHub Copilot AI credits
- Hints on sign-up process from past experience:
 - Your application has to be approved by GitHub
 - This will only work if you do the following:
 - Your profile has to be complete (full name, NHH added as institution/employer)
 - Your account must be associated with an NHH email address (primary or additional)
 - You must apply while logged into the eduroam Wi-Fi on campus
 - You must attach a screenshot of your student ID app
 - Once approved, there may be one additional step to actually claim the education voucher

Other tools

Mandatory

- Code formatting: [Ruff VS Code extension](#)

Optional

- Git/GitHub: [gh CLI](#) (Linux, macOS, Windows)
- Local docs: [Dash](#) (macOS), [Zeal](#) (Linux, Windows)
- Terminals: [kitty](#) (Linux, macOS), [Ghostty](#) (Linux, macOS)
- Shell prompts: [Starship](#) (Linux, macOS, Windows)

GOAL 2:
SET UP LOCAL TECH₂-H₂6 REPOSITORY

Forking vs. cloning

Forking: Independent copy on GitHub

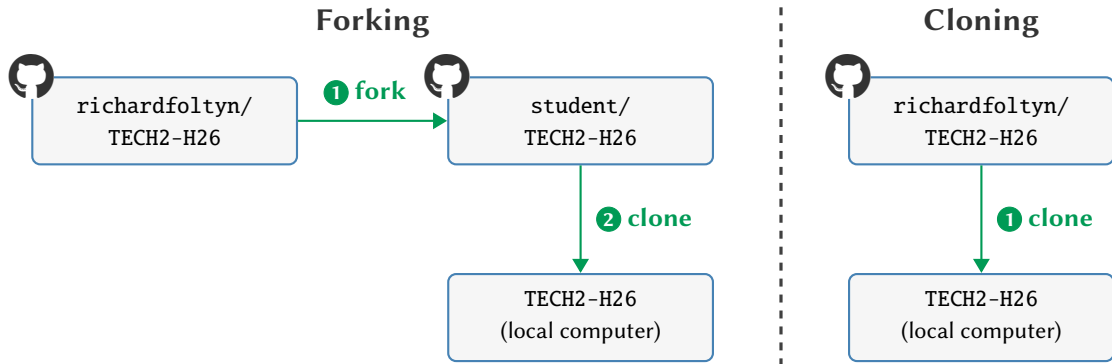
- Creates a copy under your own GitHub account
- Full write access: you can commit and push changes freely
- No automatic updates from the original repository (must sync explicitly on GitHub)
- You still need to *clone* the fork to work on your local computer

Cloning: Local copy on your computer

- Downloads a remote repository onto your local machine
- Directly linked to the remote repository
- Receive updates by running `git pull`
- You **cannot** push changes to the remote repository without write permissions

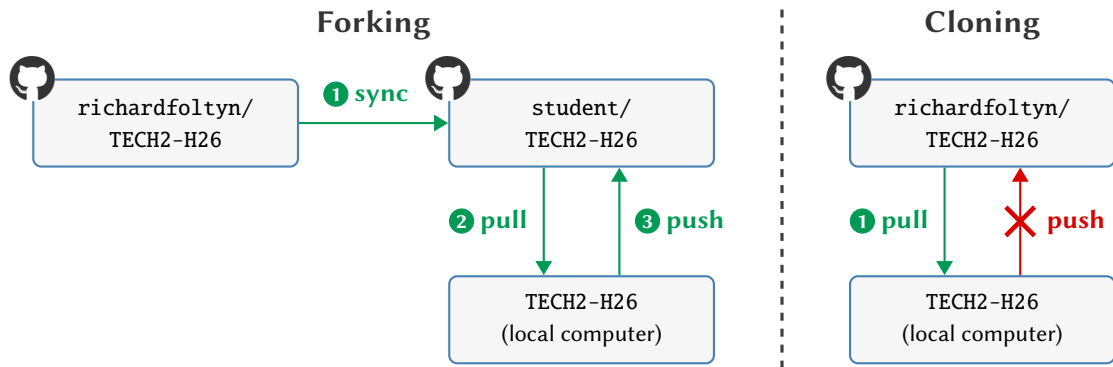
Forking vs. cloning

Initial setup



Forking vs. cloning

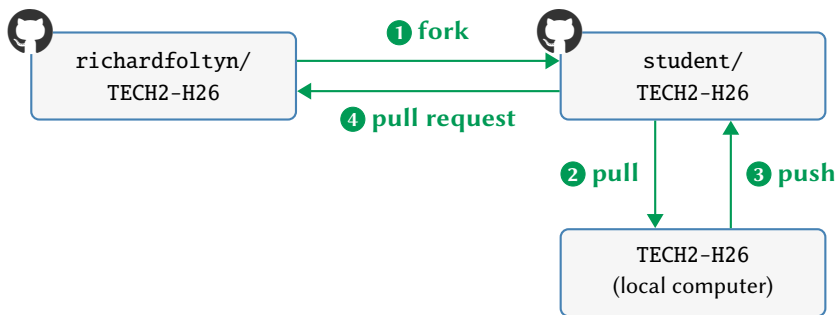
Syncing workflow



Warning: Syncing will fail if you have edited the same file as the original repository (merge conflict).

Collaboration workflow: Forking + pull requests

- It's impractical/undesirable to grant collaborators write access to your repo
- Solution: each collaborator forks the repository, works on their own copy, and proposes merges to upstream via **pull requests** (PR)



GOAL 3:
SET UP TECH2 CONDA ENVIRONMENT

Set up the TECH2 Conda environment (if you haven't done so)

Open **Anaconda Prompt** (Windows) or **Terminal** (macOS):

Option 1: In your cloned repository directory

- Navigate to the directory containing `environment.yml` and run:

```
conda env create -f environment.yml
```

Option 2: Direct download from GitHub

- Download `environment.yml` and create the environment in one step:

```
curl -O https://raw.githubusercontent.com/richardfoltyn/TECH2-H26/main/environment.yml  
conda env create -f environment.yml
```

Verify environment:

- Run [lectures/lecture1/test-environment.ipynb](#) to verify that the environment is working.

GOAL 4:
IMPLEMENT AND BENCHMARK `ARGMAX()`

Implement and benchmark `argmax()`

Motivation & background:

- Implement a function `argmax()` that takes a sequence of numbers and **returns the index** (position) of the maximum element.
- Explore performance differences: pure Python loops vs. vectorized operations in compiled libraries like NumPy.
- Benchmark the custom implementation against NumPy's `np.argmax()` across different sample sizes.

Getting started:

- Locate the template [lectures/lecture1/lecture1-template.py](#).
- **Rename** the template to `lecture1_XX.py` (where XX are your initials) to avoid merge conflicts when syncing future updates.

Implementing argmax() with loops

- Implement `argmax()` to take a sequence of numbers and return the index (position) of the maximum element.

```
def argmax(values):  
    ...  
    return imax
```

- Desired output:

```
values = [2, 3, -1, 7, 4]  
i = argmax(values)  
print(i) # prints 3
```

Benchmarking against NumPy's `argmax()` implementation

- Use `%timeit` cell magic in Jupyter notebooks to time the execution of a statement:

```
# Import your implementation of argmax (replace XX with your initials)
from lecture1_XX import argmax
```

```
%timeit argmax(values)
```

Note: This does not work in regular `.py` files because `%timeit` is not valid Python syntax.

- Compare to NumPy's implementation:

```
import numpy as np
```

```
%timeit np.argmax(values)
```

Script vs. module: `__name__ == '__main__'`

Dual role of Python files:

- A .py file can be run **directly as a script** (e.g., `python lecture1.py`), or **imported as a module** into a notebook/script (e.g., `import lecture1`).

The special variable `__name__`:

- When executed directly: Python sets `__name__ = '__main__'`.
- When imported: Python sets `__name__` to the module name (e.g., `'lecture1'`).

Guarding executable code:

```
def main():  
    ...  
  
if __name__ == '__main__':  
    main()
```

- Code inside the `if` block runs only on direct execution, **not** when imported into a Jupyter notebook.

GOAL 5:
DEBUGGING IN VS CODE

Debugging in VS Code

What is a debugger?

- Pauses program execution at specified lines (**breakpoints**) to inspect variable states in memory.
- Enables step-by-step execution to trace program flow without inserting temporary `print()` statements.
- Allows interactive evaluation of expressions in the debug console during runtime.

Debugging in VS Code:

F5	Start / Continue (launch debugger or run to next breakpoint)
F10	Step over (execute current line and advance to next)
F11 / Shift + F11	Step into / Step out of function
Shift + F5	Stop debugging
F9	Toggle breakpoint (or click margin left of line number)

On macOS, hold `fn` with function keys (e.g., `fn` + F5)

ADDITIONAL RESOURCES

Additional resources — Videos

Introduction to the command line / terminal:

- Absolute BEGINNER Guide to the **Mac OS** Terminal [17 min]
<https://youtu.be/aKRYQsKR46I>
- Git Bash - Simplest command line program for **Windows** [7 min]
<https://youtu.be/yoZ910JQzrg>

Introduction to using Git

- Git for dummies [20 min] <https://youtu.be/mJ-qvsxPHpY>
- Git and GitHub Tutorial for Beginners [46 min] <https://youtu.be/tRZGeaHPoaw>
- Git Essentials in VS Code [30 min] <https://youtu.be/twsYxYaQikI>
Focuses on interacting with git and GitHub through VS Code